Bilkent University

Department of Electrical and Electronics Engineering

EE-102: Introduction to Digital Circuit Design

Lab 6 Report: Arbitrary Waveform Generator (PWM)

Author: Can Kurc

ID: 22502100

Date: April 2, 2026

1 Introduction and Objective

The purpose of this lab was to design an Arbitrary Waveform Generator of my choosing. I decided to make a Pulse Width Modulation (PWM) signal generator using the Basys3 FPGA. The design had to generate a precise 1kHz frequency. The PWM generator was made in such a way that the duty cycle of the waveform could be dynamically controlled using the 8 slide switches on the board, which allowed variable pulse width without any timing errors, since a clock cycle error of even 10ns was not allowed.

2 Design and Architecture

The system architecture is made up of a custom VHDL module (`pwm_generator`) integrated with the Vivado Clocking Wizard through an IP Block Design.

2.1 VHDL Logic Design

The entire logic relies on a single synchronous process triggered by the 100MHz clock.

- **The Time Base:** A 17-bit counter (`refresh_counter`) counts from 0 to 99,999. This creates 100,000 clock cycles which results in a 1 ms period (1 kHz frequency).
- **The Duty Cycle:** The 8-bit switch input (`sw`) is converted to an integer and scaled up by a factor of 392 to map the 0–255 input range to the 0–100,000 counter range.
- **The Comparator:** If the counter is less than the threshold, the logic outputs HIGH. Otherwise, the output is LOW.

The counter is designed to reset at 99,999 as this creates exactly 100,000 clock cycles, resulting in a 1kHz base frequency. If the counter were allowed to overflow to its 17-bit maximum (131,071), the resulting frequency would be approximately 762Hz, which would fail the 1kHz lab requirement.

2.2 Block Design Integration

To ensure a stable clock signal, the design uses the Vivado IP Integrator. Vivado's Clocking Wizard IP was used to generate a clean, stable 100MHz clock from the Basys3's internal oscillator. This diminishes potential signal jitter and creates a reliable time base for the PWM logic. The Clocking Wizard and the PWM module were connected in the IP Integrator.

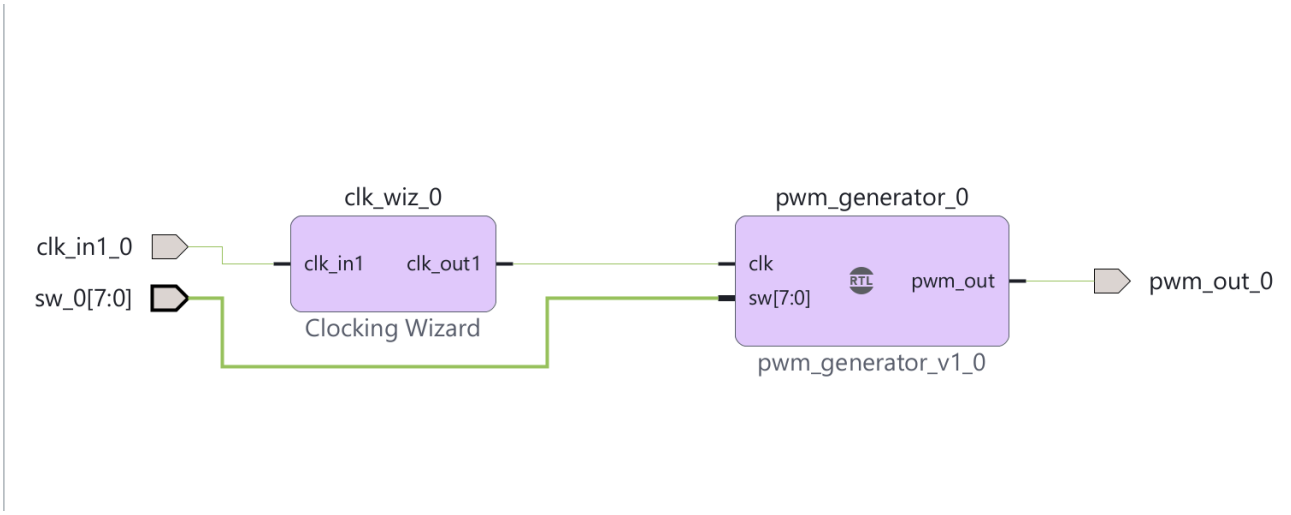


Figure 1: Vivado Block Design showing the Clocking Wizard integrated with the PWM Generator

3 Simulation Results

To simulate the 100MHz clock and verify the timing precision of the generated waveform, a test-bench was created. To ensure the waveforms are easily visible and verifiable on standard laboratory oscilloscopes, two mid-range test cases were selected.

3.1 Test Case 1: 50% Duty Cycle

The switches were set to 128 (Hex `x"80"`) and from calculations, logic high duration was expected to be:

$$128 \times 392 = 50{,}176 \text{ cycles} \quad \Rightarrow \quad 50{,}176 \times 10 \text{ ns} = 501.76 \ \mu\text{s}$$

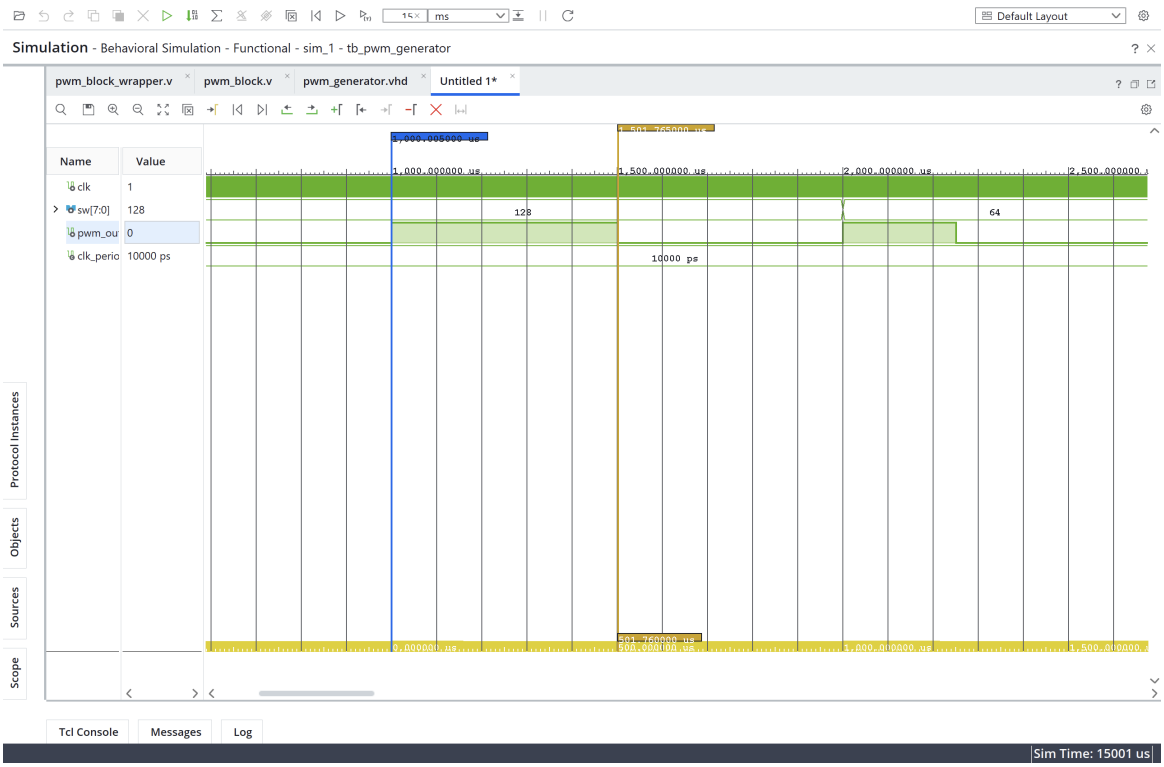


Figure 2: Vivado simulation measuring exactly 501.760 μs for the logic high, and a 1.000 ms total period

With the help of Vivado's Next Transition feature, the simulation measurement markers were snapped exactly to the rising and falling edges of the waveform. The measurement delta for the logic high time was 501.760 μs and total period was 1.000 ms according to the simulation.

3.2 Test Case 2: 25% Duty Cycle

The switches were set to 64 (Hex `x"40"`) and from calculations, logic high duration was expected to be:

$$64 \times 392 = 25{,}088 \text{ cycles} \quad \Rightarrow \quad 25{,}088 \times 10 \text{ ns} = 250.88 \ \mu\text{s}$$

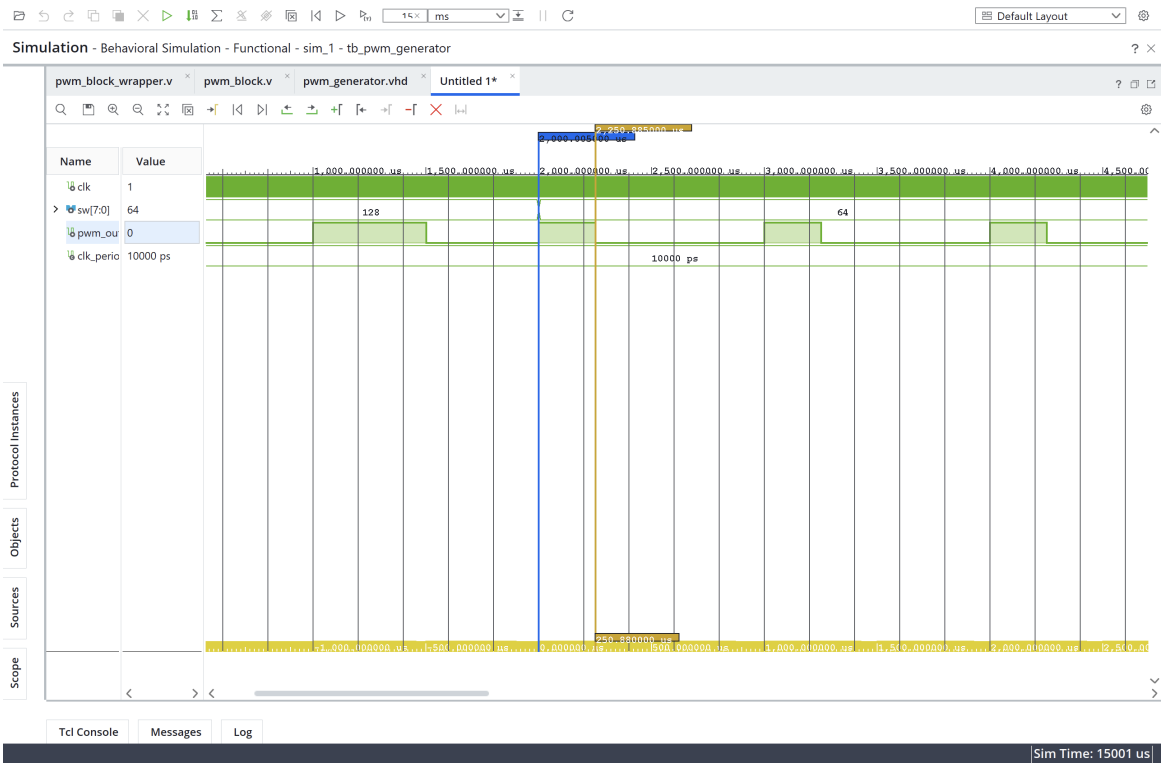


Figure 3: Vivado simulation showing the pulse width dropping to 250.880 μs while the 1 ms period remains constant

Halving the input switch value from 128 to 64 caused a high time of 250.880 μs, which is half of the previous duty cycle. With this measurement, it was confirmed that the variable scaling logic (`to_integer(unsigned(sw)) * 392`) maps the 8-bit user input (0–255) into the appropriate active-high cycle count out of the 100,000 total cycles of the period.

4 Hardware Implementation

4.1 Constraints Mapping

The external ports from the Block Design wrapper were mapped to the physical hardware:

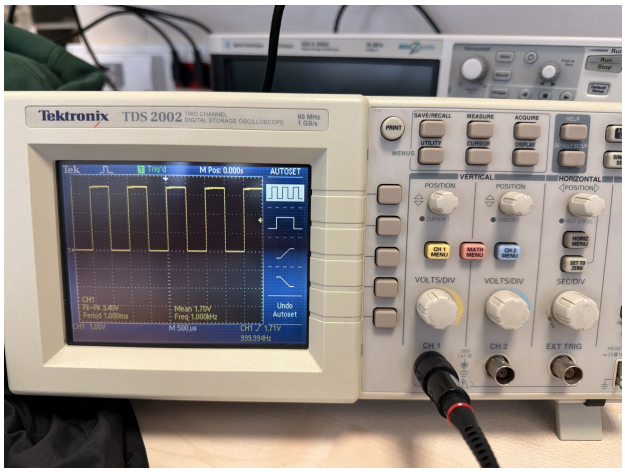
- `clk_in1_0` to `W5` (100 MHz clock).
- `sw_0[7:0]` to the 8 physical slide switches.
- `pwm_out_0` to Pin `J1` (Pmod Header JA, Pin 1).

4.2 Oscilloscope Verification

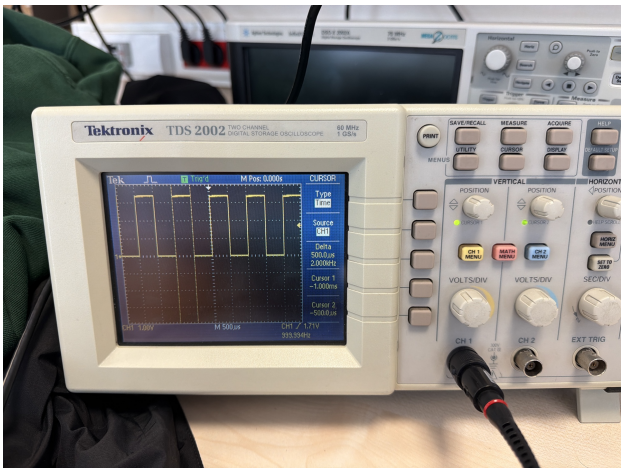
The output was probed on the oscilloscope to confirm the simulation results translated correctly to the hardware. The two same test cases from the simulation were recreated physically.

4.2.1 Hardware Test Case 1: 50% Duty Cycle

With only SW7 flipped UP (value 128), the oscilloscope captured the resulting square wave.



(a) Oscilloscope reading verifying the 1.000 kHz base frequency and 1.000 ms period



(b) Oscilloscope reading for SW=128, measuring a 500.0 μs logic high pulse width

Figure 4: Hardware Verification for 50% Duty Cycle

The oscilloscope measurements validated the theoretical values with the simulated ones. The baseline signal frequency remained at a steady 1 kHz with a 1ms period, which proved that the 99,999-counter limit worked flawlessly in the hardware. Measuring the duty cycle using the oscilloscope's time cursors read the logic high duration as 500.0 μs. This closely matches the theoretical 501.76 μs calculation, confirming the logic.

4.2.2 Hardware Test Case 2: 25% Duty Cycle

With only SW6 flipped UP (value 64), the duty cycle was visibly reduced on the screen.

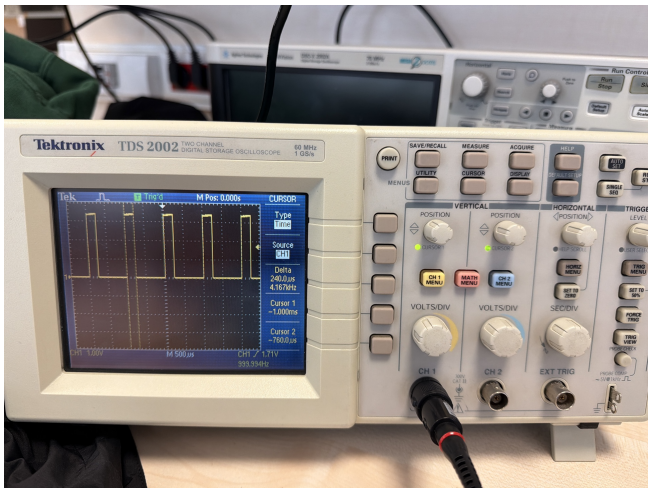


Figure 5: Oscilloscope reading for SW=64, showing a measured 240.0 μs pulse width

After changing the switch input to 64, the logic high duration shrank on the oscilloscope screen as well. A 250.88 μs pulse width was expected from the simulation and a 240.0 μs delta was measured from the oscilloscope cursors. This small deviation is not a flaw in the hardware logic but a limitation of digital quantization and pixel resolution limits of the oscilloscope.

5 Conclusion

The Arbitrary Waveform Generator was successfully designed, simulated, and deployed onto the Basys 3 FPGA hardware. By utilizing a fully synchronous clock domain and precise mathematical limits (resetting the counter at exactly 99,999), the design bypassed the off-by-one timing and 10ns precision errors cautioned against in the lab manual. The integration of Vivado's IP Integrator and Clocking Wizard ensured a reliable and clean time base. Ultimately, the physical oscilloscope verification using the Tektronix TDS 2002 demonstrated that mathematically precise simulation timings are reliably translated into high-fidelity physical signals, accounting for expected display resolution limits in measuring equipment.

6 Appendix: Source Code

6.1 pwm_generator.vhd

Listing 1: pwm_generator.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity pwm_generator is
    Port ( clk : in STD_LOGIC;
           sw : in STD_LOGIC_VECTOR (7 downto 0);
           pwm_out : out STD_LOGIC);
end pwm_generator;

architecture Behavioral of pwm_generator is
    signal refresh_counter : STD_LOGIC_VECTOR (16 downto 0) := (others => '0');
begin
    process(clk)
        variable v_threshold : integer := 0;
    begin
        if rising_edge(clk) then
            v_threshold := to_integer(unsigned(sw)) * 392;

            if unsigned(refresh_counter) = 99999 then
                refresh_counter <= (others => '0');
            else
                refresh_counter <= std_logic_vector(unsigned(refresh_counter) + 1);
            end if;

            if unsigned(refresh_counter) < v_threshold then
                pwm_out <= '1';
            else
                pwm_out <= '0';
            end if;
        end if;
    end process;
end Behavioral;
```

6.2 tb_pwm_generator.vhd

Listing 2: tb_pwm_generator.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_pwm_generator is
end tb_pwm_generator;

architecture Behavioral of tb_pwm_generator is
    component pwm_generator
        Port ( clk : in STD_LOGIC;
               sw : in STD_LOGIC_VECTOR (7 downto 0);
               pwm_out : out STD_LOGIC);
    end component;

    signal clk     : STD_LOGIC := '0';
    signal sw      : STD_LOGIC_VECTOR (7 downto 0) := (others => '0');
    signal pwm_out : STD_LOGIC;

    constant clk_period : time := 10 ns;
```

```
begin
    uut: pwm_generator port map (
        clk => clk,
        sw => sw,
        pwm_out => pwm_out
    );

    clk_process : process
    begin
        clk <= '0';
        wait for clk_period/2;
        clk <= '1';
        wait for clk_period/2;
    end process;

    stim_proc: process
    begin
        -- Test Case 1: 50% Duty Cycle
        -- 128 * 392 = 50,176. Expected High Time: 501.76 us.
        sw <= x"80";
        wait for 2 ms;

        -- Test Case 2: 25% Duty Cycle
        -- 64 * 392 = 25,088. Expected High Time: 250.88 us.
        sw <= x"40";
        wait for 2 ms;

        wait;
    end process;
end Behavioral;
```

6.3 constraints.xdc

Listing 3: constraints.xdc

```
## Clock signal (100 MHz)
set_property PACKAGE_PIN W5 [get_ports clk_in1_0]
    set_property IOSTANDARD LVCMOS33 [get_ports clk_in1_0]
    create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports
        clk_in1_0]

## Switches (sw_0[7:0])
set_property PACKAGE_PIN V17 [get_ports {sw_0[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {sw_0[0]}]
set_property PACKAGE_PIN V16 [get_ports {sw_0[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {sw_0[1]}]
set_property PACKAGE_PIN W16 [get_ports {sw_0[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {sw_0[2]}]
set_property PACKAGE_PIN W17 [get_ports {sw_0[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {sw_0[3]}]
set_property PACKAGE_PIN W15 [get_ports {sw_0[4]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {sw_0[4]}]
set_property PACKAGE_PIN V15 [get_ports {sw_0[5]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {sw_0[5]}]
set_property PACKAGE_PIN W14 [get_ports {sw_0[6]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {sw_0[6]}]
set_property PACKAGE_PIN W13 [get_ports {sw_0[7]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {sw_0[7]}]

## PWM Output (Pmod Header JA, Pin 1)
set_property PACKAGE_PIN J1 [get_ports pwm_out_0]
    set_property IOSTANDARD LVCMOS33 [get_ports pwm_out_0]
```

```
27
28 ## Configuration bits for Basys 3
29 set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
30 set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
31 set_property CONFIG_MODE SPIx4 [current_design]
```